McCAIG INSTITUTE
FOR BONE AND JOINT HEALTH

Git Use And Software Organization

3rd ORMIR Workshop, July 9, 2026

Erica Baldesarra

Mobility for Life.

UNIVERSITY OF CALGARY

Utilizing Git (Version Control)

ALL HAIL PRO GIT

git status

git add

git commit [-m]

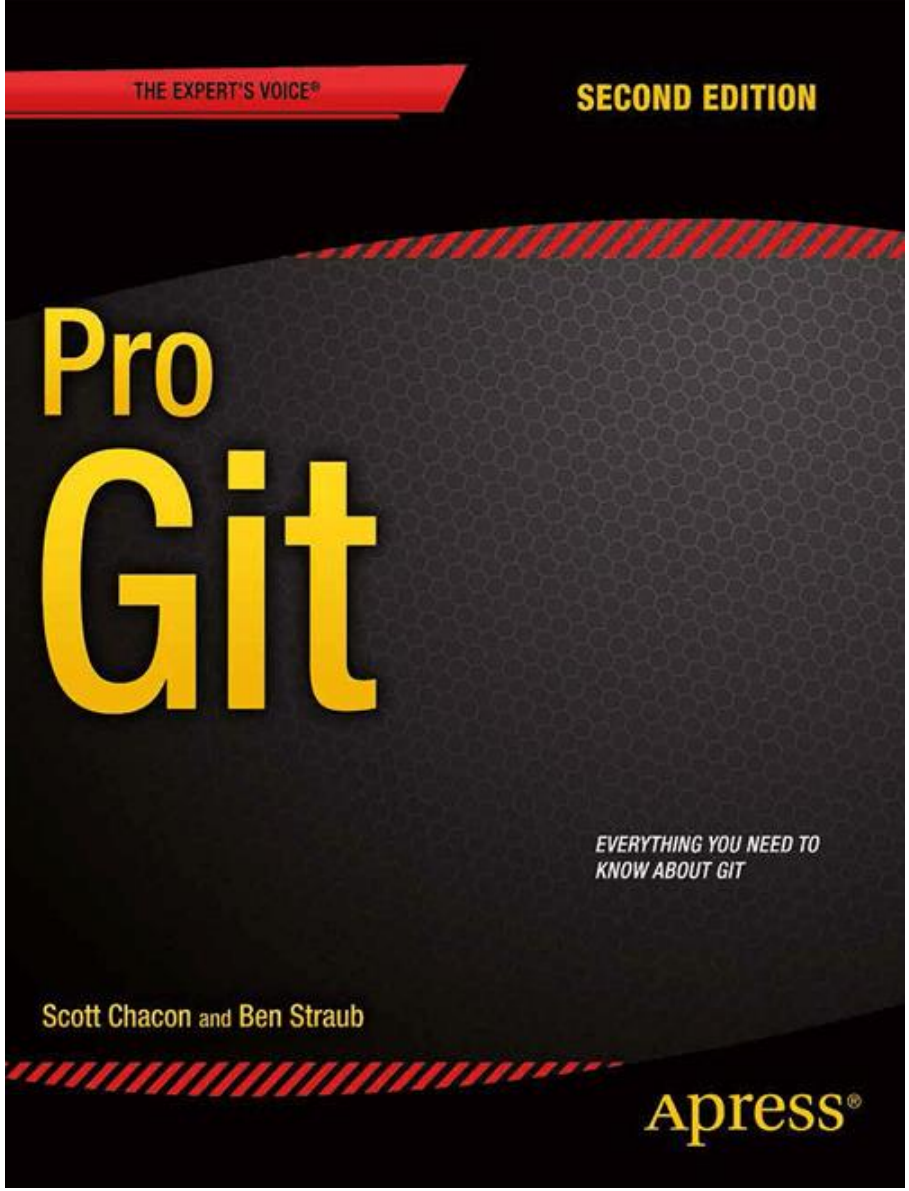
git checkout [-b]

git push

git pull

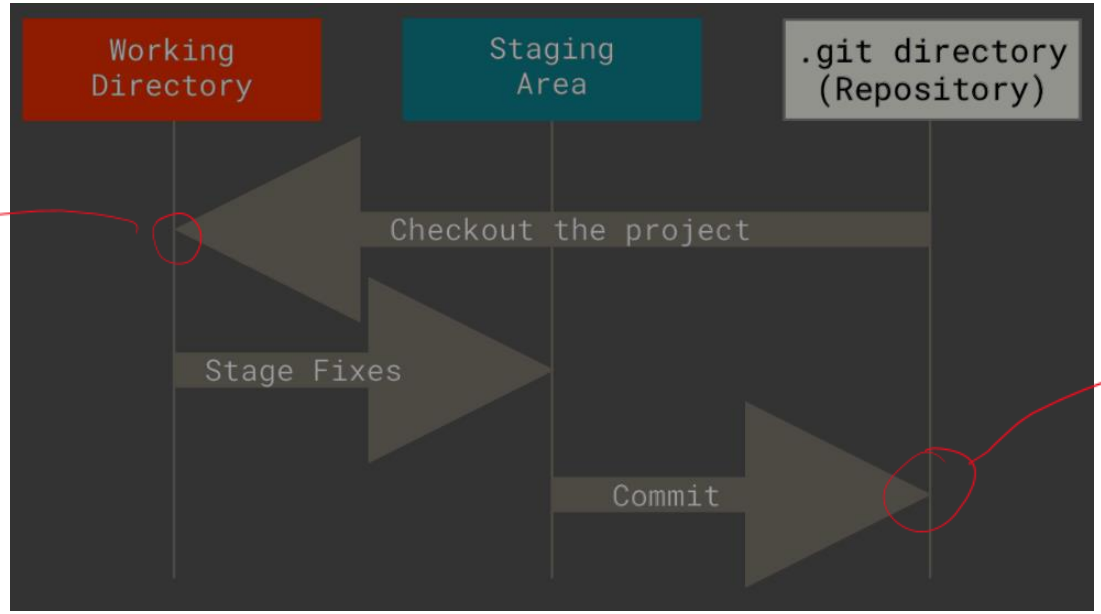
git log

McCAIG INSTITUTE
FOR BONE AND JOINT HEALTH



Git: What even is it?

Version Control System: “A stream of snapshots”



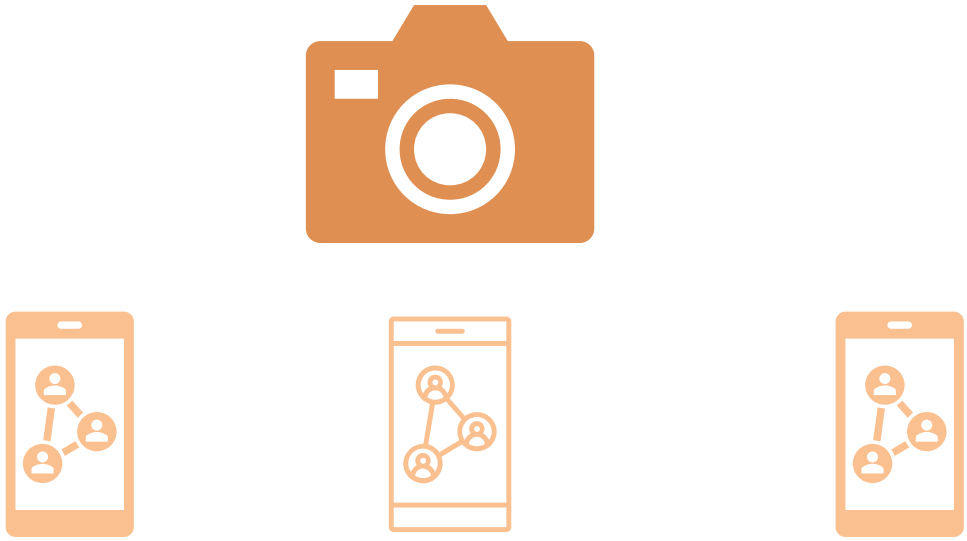
Collect Previous Snapshot

Create New Snapshot!

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Git Providers (Local vs Cloud-based)

Think: Camera -> Social Media

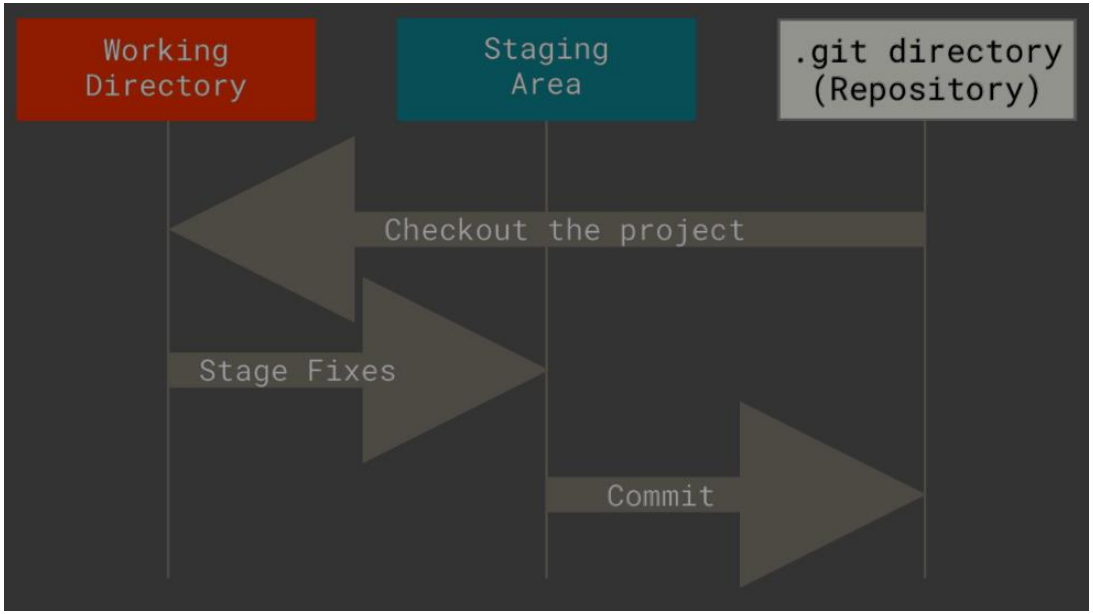


McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Kiron, Toufiq Hasan. *Toufiq Hasan Kiron*. kiron.dev/blogs/managing-multiple-git-providers-and-syncing-changes-simultaneously.

Concept: Local vs Remote Changes

Local



Remote

Bitbucket

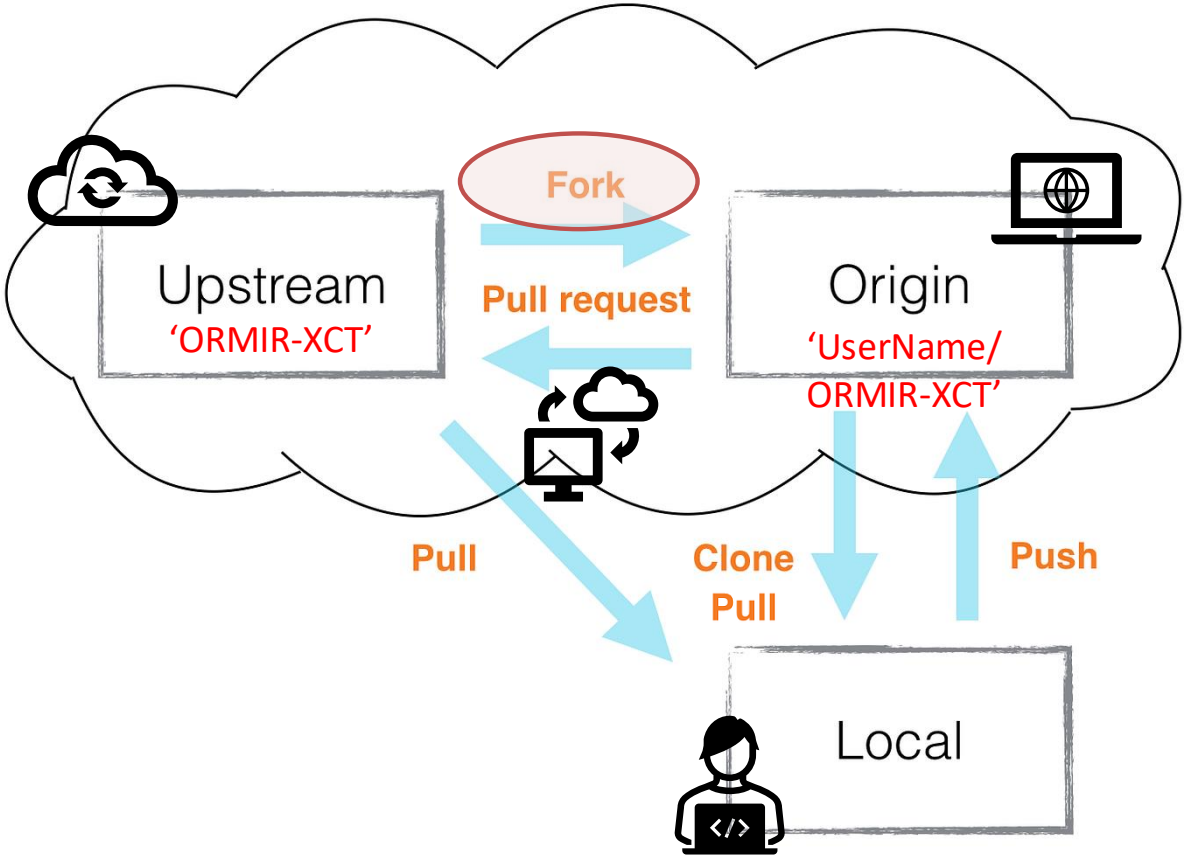
GitLab

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Chacon, Scott, and Ben Straub. *Pro Git*. 2nd ed., Apress, 2014. *Git SCM*, https://git-scm.com/book/en/v2.
Kiron, Toufiq Hasan. *Toufiq Hasan Kiron*. kiron.dev/blogs/managing-multiple-git-providers-and-syncing-changes-simultaneously.

Forking (Online/Remote)

Adding to someone else's repository

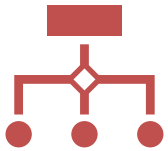


McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Verma, Pragati. *Medium*, 24 Mar. 2020, levelup.gitconnected.com/confusing-terms-in-the-git-terminology-c7115d6febc7.

Utilizing git

Branches/Pull Requests



Sub-divide groups of changes onto feature branches

Be as messy as you'd like there!

Only once you are sure you have polished and tested your work should you put it in the main branch

Don't commit the cardinal sin: don't push to main (or master) branch

Utilize PR's

Group and summarize commits as a **merge** or **squash**

McCAIG INSTITUTE
FOR BONE AND JOINT HEALTH


GIT PUSH --FORCE
PROBLEM SOLVED
makeameme.org


$ git push origin master --force


YOU SHALL NOT PUSH


GIT PUSH ORIGIN MASTER --FORCE


YOUR HONOR
MY CLIENT DIDN'T KNOW THEY WERE PUSHING TO THE MAIN BRANCH


checkout feature branch
push changes to branch
they get pushed to main
they get pushed to main

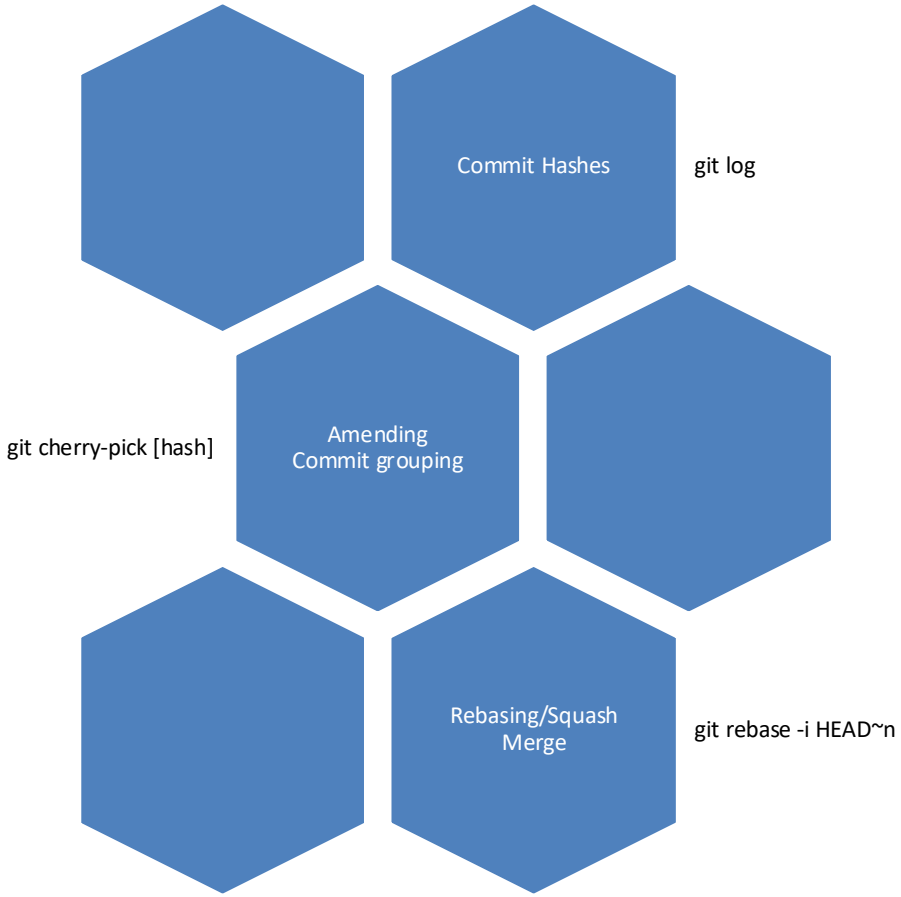

Is git push -f bad?
Depends on the context
On your branch? No
On master? Yes!

Utilizing git

Commits/Log History

Your code should speak for itself	Git stores history of all code changes If you cannot sum up your commit in 1-2 sentences, it's too big! (likely)
Action languge:	Add Update Fix
Specify type:	Bugfix Feature Style/documentation

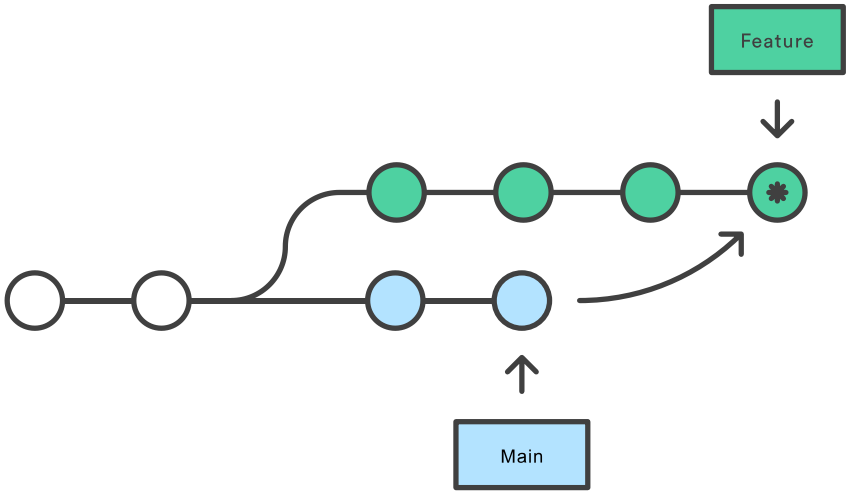
Clean History Practices



McCAIG INSTITUTE
FOR BONE AND JOINT HEALTH

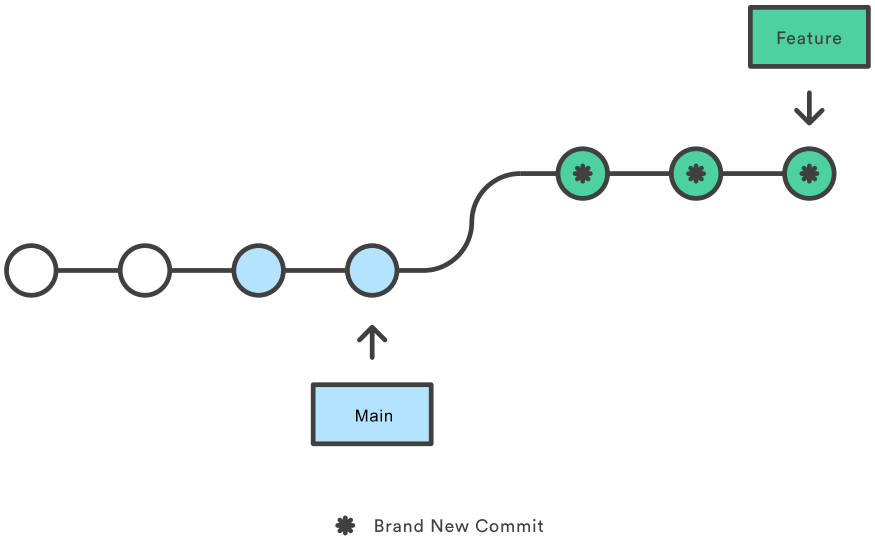
Git Merge vs Git Rebase

Merging main into the feature branch



Merges and maintains both branch histories, but adds merge commit

Rebasing the feature branch onto main

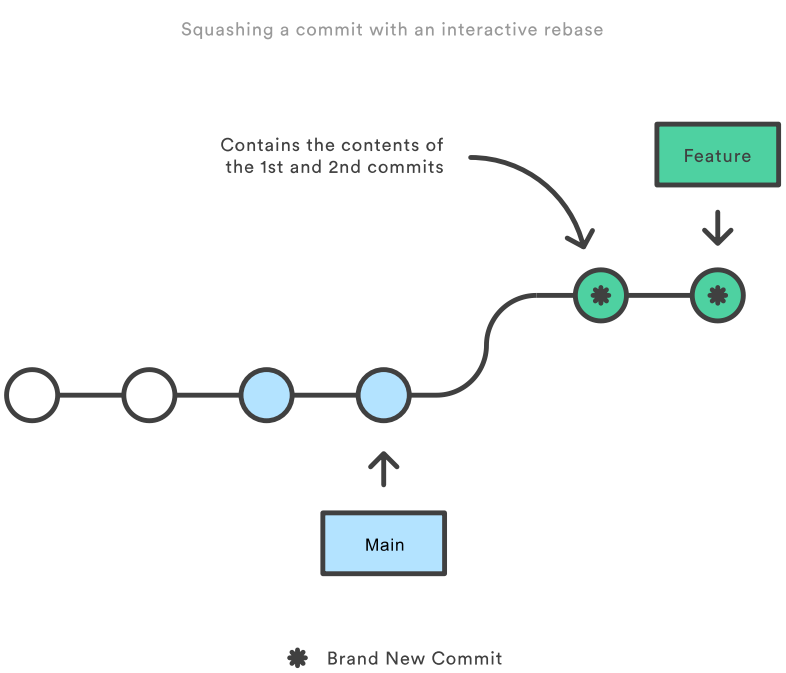


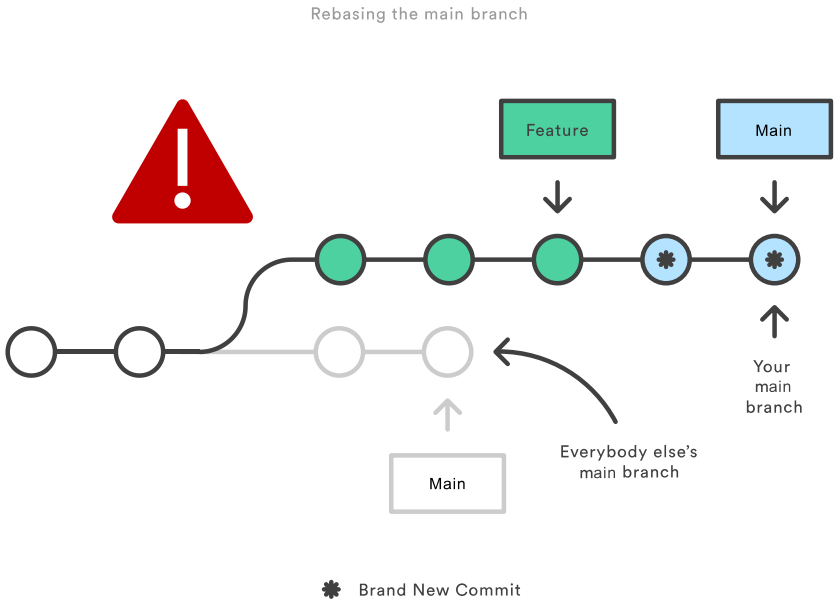
Cleaner, but more ‘dangerous’ and less context

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

https://www.atlassian.com/git/tutorials/merging-vs-rebasing

Rebasing






```
git rebase -i main
```

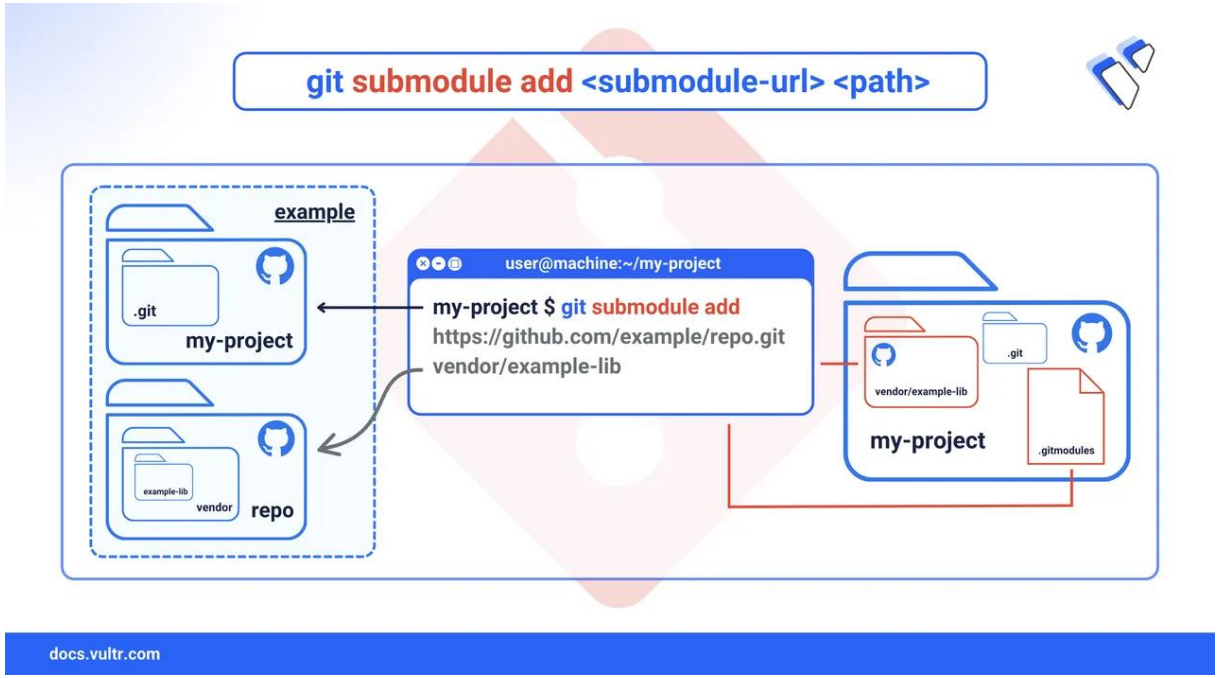
NEVER rebase a public branch, only your feature branch

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

https://www.atlassian.com/git/tutorials/merging-vs-rebasing

Submodules

Don’t re-invent the wheel!



Be cautious of licences.. You may need to carry them into your project

Releases

Publishing code: Release Packages, tags

“A lightweight tag is very much like a branch that doesn’t change — it’s just a pointer to a specific commit.”

> **Sharing Tags**
>
> By default, the `git push` command doesn’t transfer tags to remote servers. You will have to explicitly push tags to a shared server after you have created them. This process is just like sharing remote branches — you can run `git push origin <tagname>`.

Utilize **continuous integration** where you can (GitHub Actions)

Git Resources/References

ORMIR GitHub Contributing guidelines: https://www.ormir.org/code_guidelines/gh-contributing/

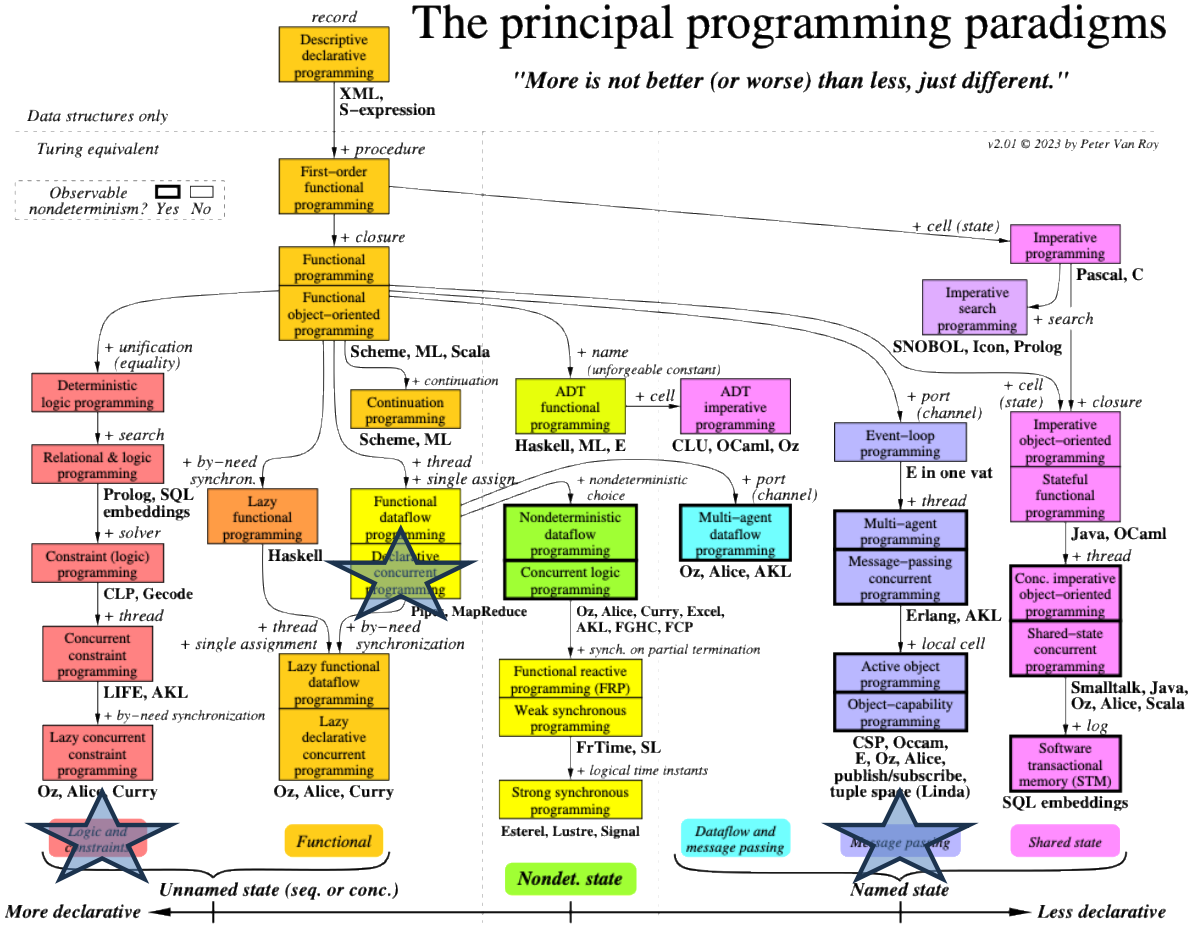
Command Cheat Sheet: https://git-scm.com/cheat-sheet

Pro git: https://git-scm.com/book/en/v2

Atlassian Git Tutorial: https://www.atlassian.com/git/tutorials/

Your Peers! (StackOverflow): https://stackoverflow.com/questions/tagged/github?tab=Newest

Programming Paradigms



Explanations

A *programming paradigm* is an approach to programming a computer based on a coherent set of principles or a mathematical theory. See the book *Concepts, Techniques, and Models of Computer Programming*.

The chart classifies programming paradigms according to their kernel languages (the small core language in which all the paradigm's abstractions can be defined). Kernel languages are ordered according to the creative extension principle: a new concept is added when it cannot be encoded with only local transformations. Two languages that implement the same paradigm can nevertheless have very different "flavors" for the programmer, because they make different choices about what programming techniques and styles to facilitate. All paradigms that support functional programming can also support object–oriented programming.

When a language is mentioned under a paradigm, it means that part of the language is intended (by its designers) to support the paradigm without interference from other paradigms. It does not mean that there is a perfect fit between the language and the paradigm. It is not enough that libraries have been written in the language to support the paradigm. The language's kernel language should support the paradigm. When there is a family of related languages, usually only one member of the family is mentioned to avoid clutter. The absence of a language does not imply any kind of value judgment.

State is the ability to remember information, or more precisely, to store a sequence of values in time. Its expressive power is strongly influenced by the paradigm that contains it. We distinguish four levels of expressiveness, which differ in whether the state is unnamed or named, deterministic or nondeterministic, and sequential or concurrent. The least expressive is functional programming (threaded state, e.g., DCGs and monads: unnamed, deterministic, and sequential). Adding concurrency gives declarative concurrent programming (e.g., synchrocells: unnamed, deterministic, and concurrent). Adding nondeterministic choice gives concurrent logic programming (which uses stream mergers: unnamed, nondeterministic, and concurrent). Adding ports or cells, respectively, gives message passing or shared state (both are named, nondeterministic, and concurrent). Nondeterminism is important for real–world interaction (e.g., client/server). Named state is important for modularity.

Axes orthogonal to this chart are typing, aspects, and domain–specificity. Typing is not completely orthogonal: it has some effect on expressiveness. Aspects should be completely orthogonal, since they are part of a program's specification. A domain–specific language should be definable in any paradigm (except when the domain needs a particular concept).

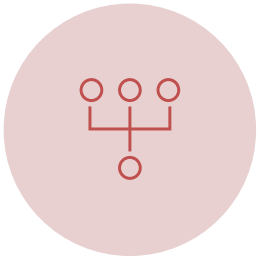
Metaprogramming is another way to increase the expressiveness of a language. The term covers many different approaches, from higher–order programming, syntactic extensibility (e.g., macros), to higher–order programming combined with syntactic support (e.g., meta–object protocols and generics), to full–fledged tinkering with the kernel language (introspection and reflection). Syntactic extensibility and kernel language tinkering in particular are orthogonal to this chart. Some languages, such as Scheme, are flexible enough to implement many paradigms in almost native fashion. This flexibility is not shown in the chart.

New Computational Paradigms for Computer Music, G. Assayag and A. Gerzso (eds.), IRCAM/Delatour France, 2009

Coding Paradigms – Know what purpose your code serves!

SHELL SCRIPTING

- Set up batch scripting
- Data input/output pre/post-processing
- Command piping
- File system manipulation



CONCURRENT PROGRAMMING

- e.g., Cloud or Server based computing
- Computationally/Memory-intensive (e.g., Registration)
- Difficult to do well!

FUNCTIONAL

- Set input -> set output
- Mathematical calculations
- Repetitive tasks with same expected output

OBJECT-ORIENTED

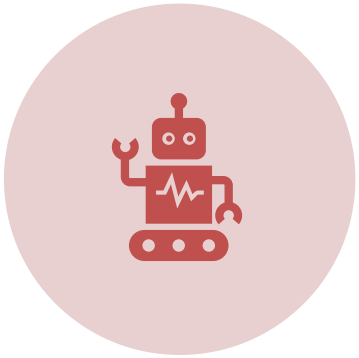
- Complicated interactions
- 'Stateful'

Don't be scared to mix and match to fit your needs! These are not one-size-fits all

Programming Paradigms

READABILITY/USABILITY

MAINTAINABILITY



SCALABILITY (THINK: RE-USEABILITY/FUTURE-PROOFING)

Q: What if I have already started and these goals aren't met? A: REFACTOR (on a separate branch ;)!

-> The benefit of Git is the old version will *always* be available

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Readability/Usability

Follow the ORMIR guidelines for…

Ideal code changes should ‘speak for themselves’ so your commit messages can be small

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

https://www.ormir.org/guidelines.html

Readability/Usability

Naming Conventions: Stick to it!

```
snake_case (python)
camelCase (C++)
PascalCase(C++)
```

```
a = 2 # 'Magic' numbers should be avoided at all costs!
# if they are universal constants, they are usually available in a library..
# if not, make sure to document them
pi = 3.14
# e.g if there is a standard scaling value on all pixels
scaling_default = 0.2 # default scale value applied to all pixels

f = 100
b = a # what was a again? what is b supposed to be? or f?

fruit = 100
apples = 2
bananas = fruit - apples # Ah! this speaks for itself.
```

ORMIR:

Naming Style

- Variables, functions, methods, modules, packages:
  `lower_case_with_underscores`
- Constants:
  `ALL_CAPS_WITH_UNDERSCORES`
- Classes and exceptions:
  `CapWords`

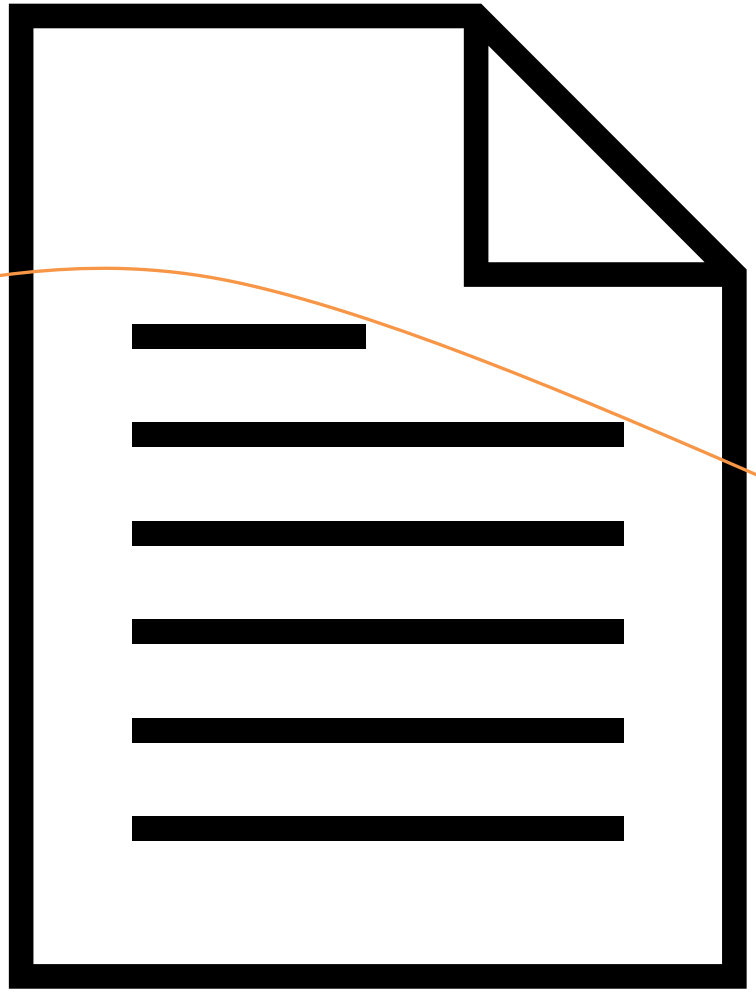
McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

https://www.ormir.org/guidelines.html

Readability/Usability

File Organization (python)

See function_ordering.py



Header Comment

Imports/Includes

Global Variables (ideally in separate module)

Function Definitions (dependencies FIRST)

Main function

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Maintainability

Modularity/Decomposition

Break-down the problem into related sub-problems

Features (e.g. visualization)	Objects (e.g. images)	Functionality (e.g. I/O)

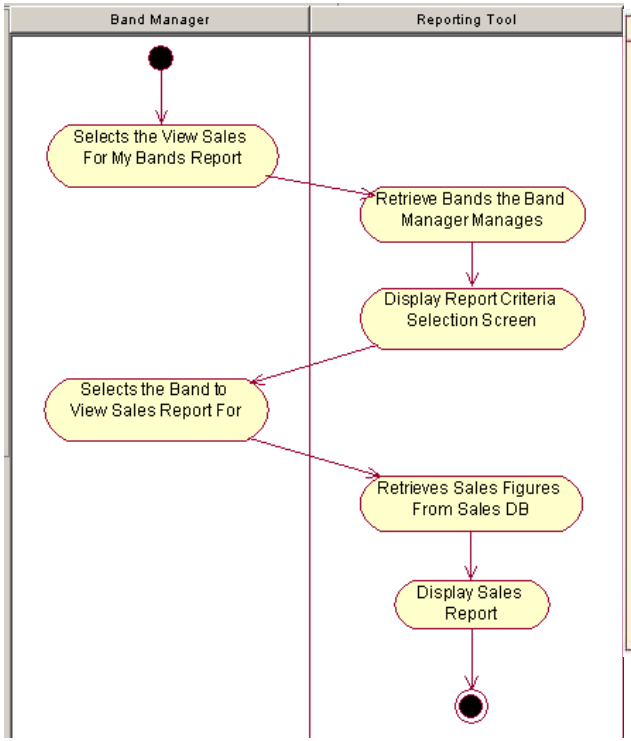
Abstract nitty-gritty details away

Sub-problems **Interface** with each other, not intertwine

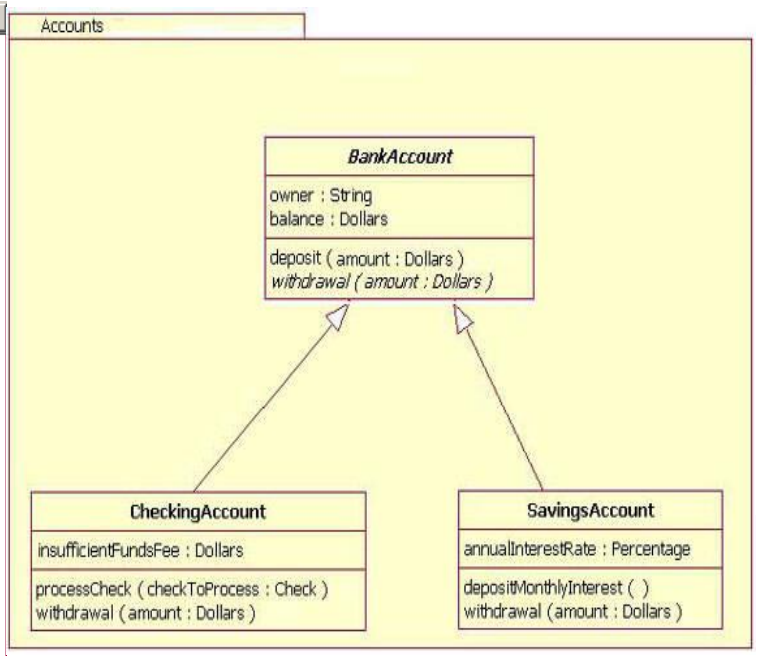
Maintainability

Diagrams

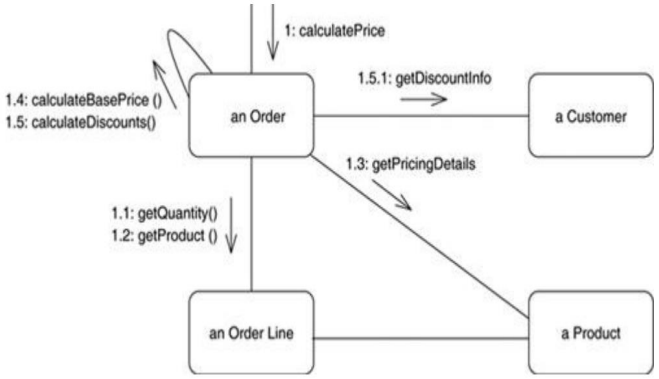
Activity/Flow/Sequence



Class/Package



Collaboration



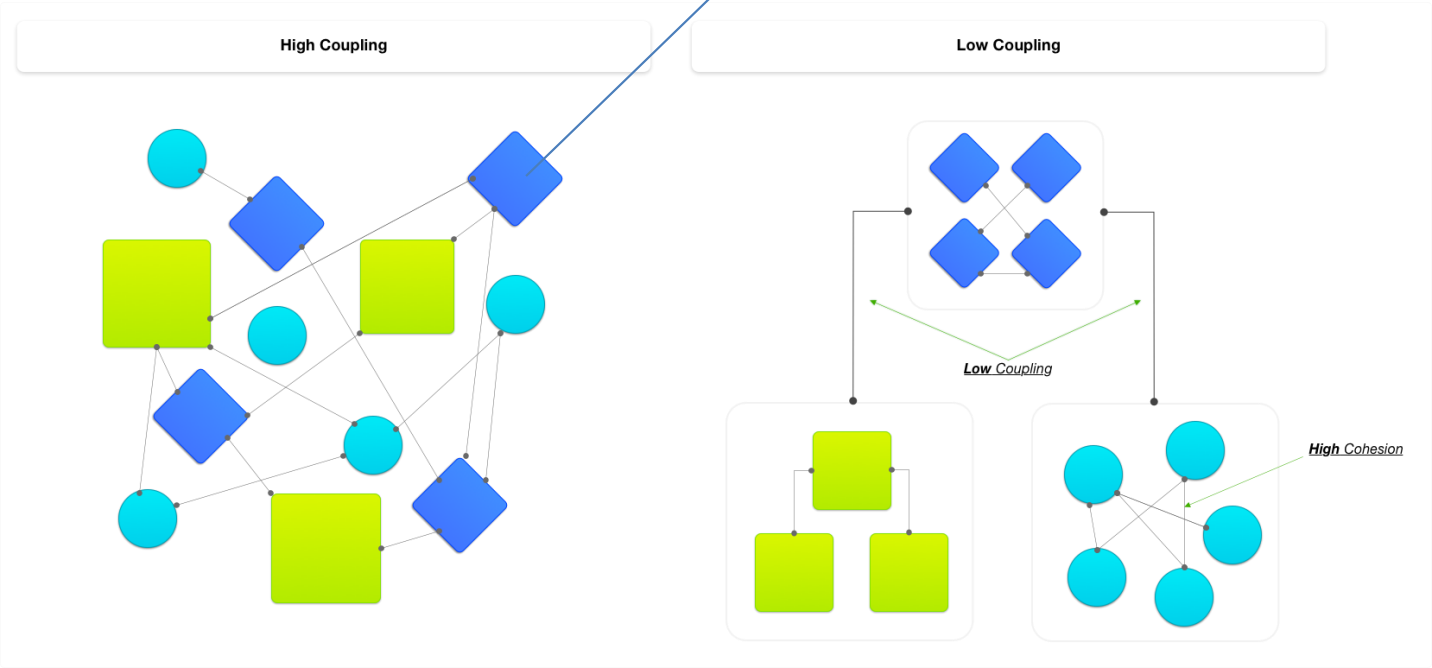
McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Donald Bell, *IBM Developer*, 2023
Fowler, *UML Distilled, 3rd Ed*, 2003

Maintainability

NOTE: many of these may be external library dependencies.. Utilize code that already exists!

High Cohesion, Low Coupling



Draw a diagram of your modules and their interactions.. If it looks too much like the left, see if you can re-arrange it

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Maintainability

Decomposition isn't always great

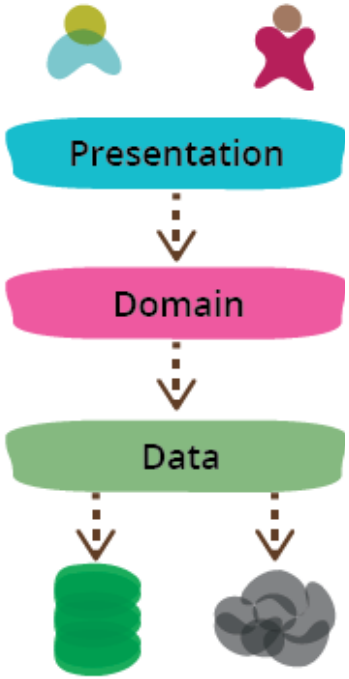
- **Decomposition can work well:**
  - E.g. designing a restaurant menu

- **Decomposition doesn't always work**
  - E.g. writing a play:

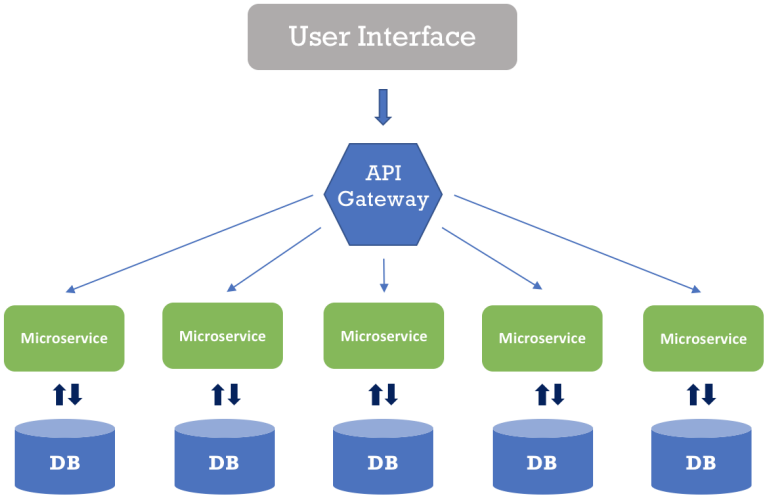
Maintainability

Design Patterns

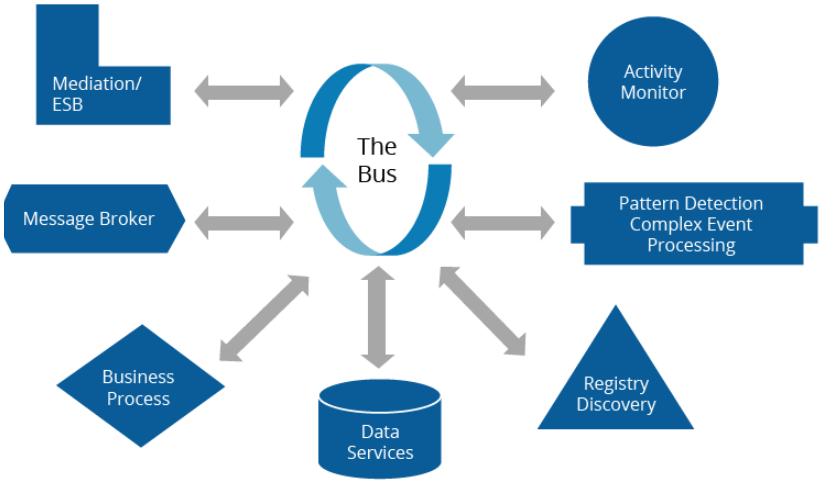
Layered



Microservices



Event-Driven Architecture



McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Fowler, 2015
Singh, *NashTechGlobal*, 2022
Abeysinghe, *WSO2*, 2016

Scalability

Functions (D.R.Y.)

D: Don't

R: Repeat

Y: Yourself

Test, Test, Test

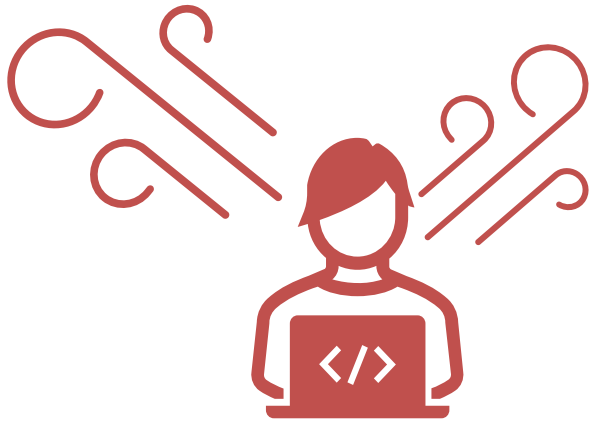
Utilize optional arguments or list-based argument groups

Example: scalability/greetings.py

McCAIG INSTITUTE FOR BONE AND JOINT HEALTH

Code Smells

- Code Duplication
- Hardcode/Magic numbers
- Bloating Classes/Functions
- ‘What’ comments
- Big fat liar
- Insider Trading



McCAIG INSTITUTE FOR BONE AND JOINT HEALTH